SMU Singapore Management University
School of Computing and Information Systems

Computational Thinking with Python

Jean Chen **James Koh**

{jeanchen, jameskohby}@smu.edu.sg

© MITB, SMU

Decomposition

© MITB, SMU

Learning Objectives

- Explain how to apply a systematic approach to problem-solving
- Learn and apply decomposition as a problem-solving strategy
- Learn and utilize different containers to tackle different problems

© MITB, SMU

How to Solve a Problem?

An early example of a systematic approach to problem-solving (Pólya, 1973):

- Understand the Problem
- Devise a Plan

The Focus of this Lecture

- Execute the Plan

Coding

- Review and Extend

Debugging, Refactoring

© MITB, SMU

Understanding the Problem

- If someone else gave it to you, try restating the problem in your own words.
- Try and represent the problem using pictures and diagrams. Humans deal better with visual representations.
- There will be knowns and unknowns at the start. You should ensure that enough information is known for you to form a solution. If there isn't, make the unknowns explicit.
- If you understand the problem, you should be able to give a pair of non-trivial input and expected output

© MITB, SMU

Devise a Plan

- A **heuristic** is a specific strategy in problem-solving that usually yields a good enough answer.

- Examples:
  - Trial and Error
  - Using a Rule of Thumb
  - Drawing an Analogy
  - Divide-and-Conquer

© MITB, SMU

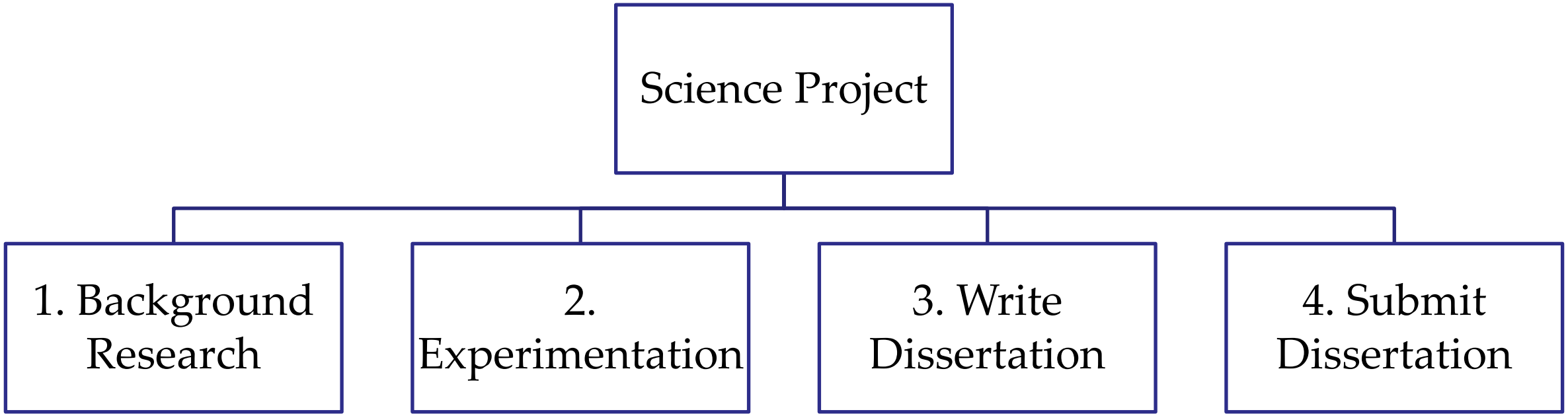
Decomposition

- A **core practice** of Computational Thinking.
- Break a complex problem down into simpler parts.
- Focus on each part and deal with it more effectively.
- Multiple tools in Python to do this:
  - If-else statements (Week 1): Handle multiple cases
  - Loops (Week 3): Handle repeated steps / patterns
  - Functions (Week 5): Organize smaller components
  - Classes / Object Oriented Programming (Week 6): Organize larger components / objects

© MITB, SMU

Tree Structure

- A larger problem can be decomposed via a Tree Structure to smaller leaves, each with a smaller sub-problem.
- Often, this yields a **sequential** series of sub-problems.



© MITB, SMU

Decomposition: Cases

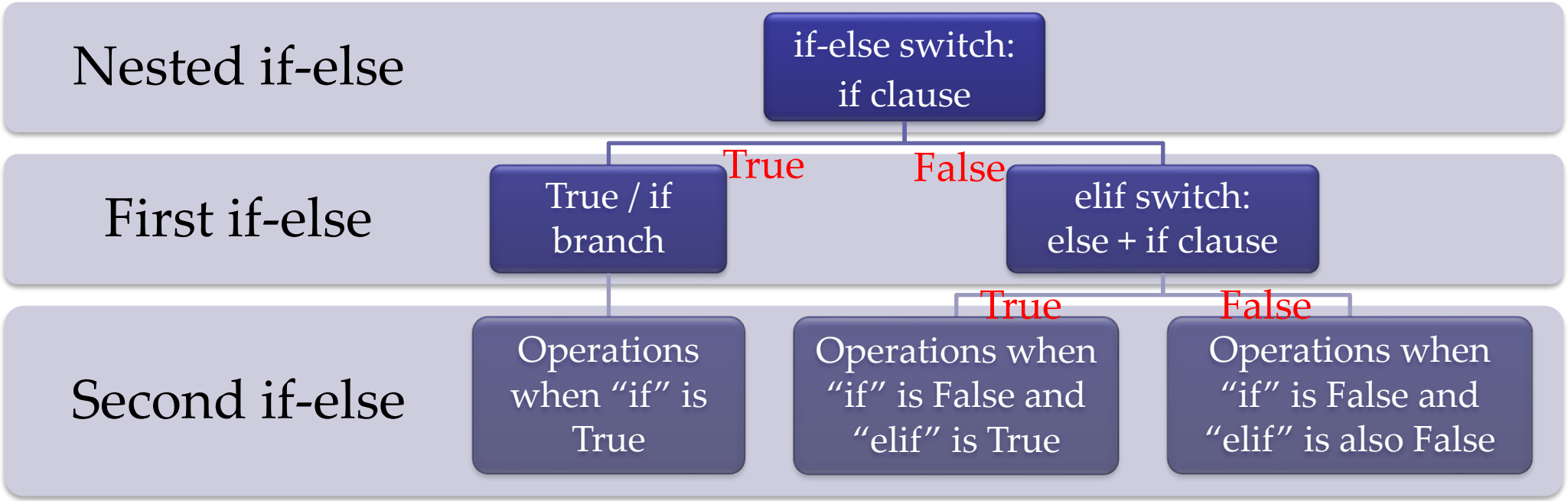
- Decomposition can also be performed to yield **parallel** subproblems
- Often, a problem can have different solutions depending on certain conditions.
- E.g., to model a traffic light junction, we have 3 cases: Red light, Orange light, and Green light
- To handle such cases, the **if-else** switch is a powerful to model different cases

```
if a test condition on variable x:
    do something when the test condition is true
else:
    do something when the test condition is false
```


© MITB, SMU

elif = else + if

- The `elif` branch allows us to include multiple cases



© MITB, SMU

Wooclap!

What is the output of the below code?

```
a = 10
if a % 5 == 0:
        print(5, end=' ')
elif a % 2 == 0:
        print(2, end=' ')
else:
        print(0, end=' ')
```

a. 5

b. 5 2

c. 5 2 0

Range and for Loops

© MITB, SMU

Range

- `range` conveniently constructs an arithmetic series
  - Often used together with list and for loops

- `range(start, stop, incr):`
  - `start` the starting value of the range, default 0
  - `stop` last value in the range should not reach or go beyond this value, cannot be omitted
  - `incr` the incremental gap between two consecutive values, default 1

© MITB, SMU

for Loops

- `for` loop processes all values in a range
- Iterates over the items in a sequence
- `for i in range(…):`
  `do something`

The variable `i` is assigned sequentially with the values in the range

The values which `i` will be assigned to

```
for i in range(5, 10):
    print(i)
```

During each step, perform the steps inside using the assigned value for `i`

© MITB, SMU

for Loops – how it works

- Always start by identifying what values can the range take
  - Here, `range(5, 10)` takes on values `5, 6, 7, 8, 9`

```
for i in range(5, 10):
    print(i)
```

excluding 10

- First Loop




```
print(i)
```

→

```
print(5)
```


© MITB, SMU

for Loops – how it works


```
for i in range(5, 10):
    print(i)
```

- Next Loop




```
print(i)
```

→

```
print(6)
```


© MITB, SMU

for Loops – how it works


```
for i in range(5, 10):
    print(i)
```

- Last Loop




```
print(i)
```

→

```
print(9)
```

- Question: What is the value of `i` **after** you finish the `for` loop?

© MITB, SMU

Exercise 2-1

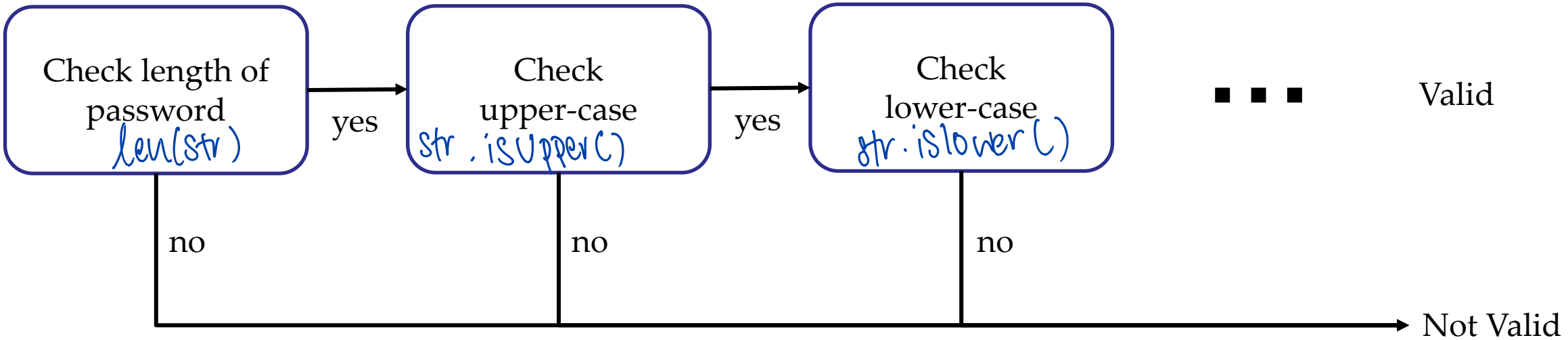
In a typical password checker, there are a set of conditions which are often checked to ensure that the password meets the required complexity of the platform. In this exercise, your job is to check whether a given password satisfies the below list of conditions:

- The password should be between length 8 to 15 (inclusive)
- The password should contain one of each of the following
  - An upper-case letter
  - A lower-case letter
  - A digit
  - A special character

© MITB, SMU

Exercise 2-1 (Solution)

- Often, a problem is given with a bunch of conditions to check
- We can use Decomposition to break this down to a sequence of subproblems
- Use flowcharts / diagrams to aid in your understanding of the problem



- Notice that the **yes** and **no** here can be interpreted as if-else conditions!

© MITB, SMU

Python Containers

© MITB, SMU

Python Containers

- Previously, we have briefly introduced Lists as Python Containers.

- Two main type of containers:
  - `list` : Lists
  - `dict` : Dictionaries
- Less commonly used:
  - `tuple` : Tuples
  - `set` : Sets

© MITB, SMU

List operations – creating a list

- Lists can be created manually using `[]`.

```
>>> ls = [1, 2, 3, 4, 5]
```

- `list()` can also be used to create lists from other sequences.

```
>>> ls = list(range(1, 6))
# creates [1, 2, 3, 4, 5]
```

- Elements of a list need not be numbers and need not be of the same type.

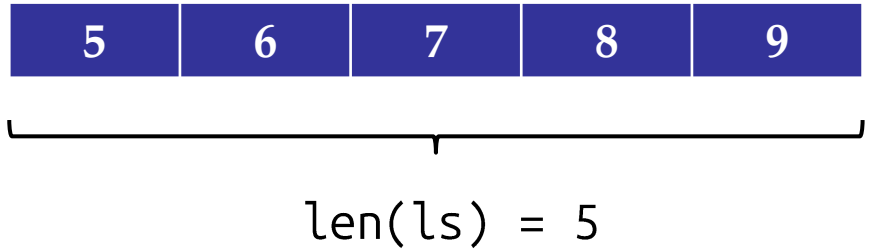
```
>>> ls = [1, '2', True, 'hello', [1, 2]]
```


© MITB, SMU

List operations – index operations

- `len(ls)`: gives the length of list `ls`

```
>>> ls = [5, 6, 7, 8, 9]
>>> len(ls)
5
```



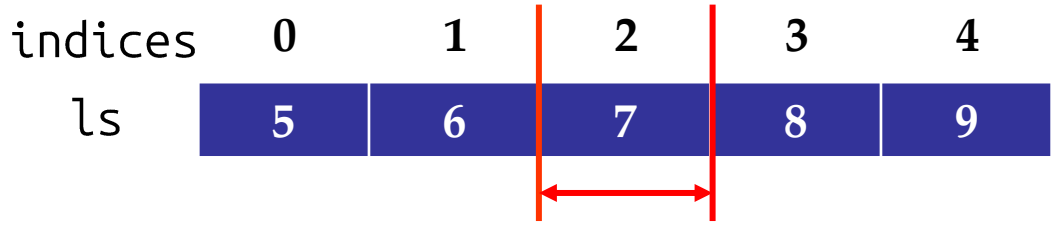
- Indices start from 0, last index is at `len(ls) - 1`



© MITB, SMU

List operations – index operations

- Slice one element
  - `list[index]`
  - e.g., `ls[2]`




```
>>> ls = [5, 6, 7, 8, 9]
>>> ls[2]
7
```


© MITB, SMU

List operations – index operations

- Slice a subset of a list
  - `list[begin:end]`: From index `begin` to `end` (exclusive of `end`)
  - `list[begin:large_number]`: To the end
  - `list[begin:]`: To the end (if `end` is omitted)
  - `list[:end]`: From beginning (if `begin` is omitted) until end (exclusive of `end`)

0 1 2 3 4

```
>>> ls = [5, 6, 7, 8, 9]
>>> ls[1:4]
[6, 7, 8]
>>> ls[1:500]
[6, 7, 8, 9]
>>> ls[1:]
[6, 7, 8, 9]
>>> ls[:4]
[5, 6, 7, 8]
```

→ start ls[1], exclude ls[4]

→ end is excessively large num also no error

→ start @ ls[1] until end.

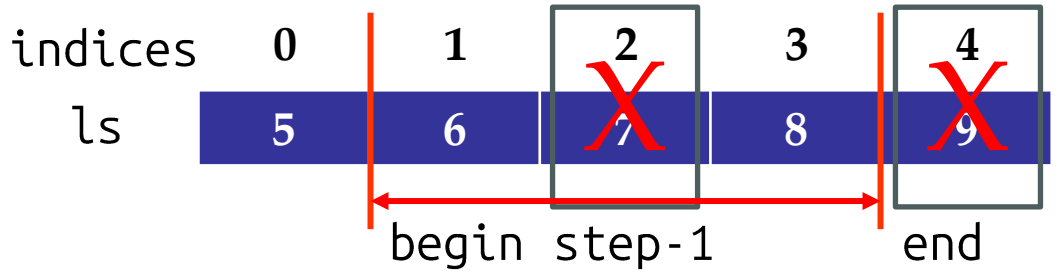
→ start @ ls[0] until ls[4] (excluding)

© MITB, SMU

List operations – index operations

- Slice a subset of a list – with step (default = 1)
  - `list[begin:end:step]`: From index `begin` to index `end`, at a `step` size, inclusive of `begin`, but exclusive of `end`

```
>>> ls = [5, 6, 7, 8, 9]
>>> ls[1:4:2]
[6, 8]
```

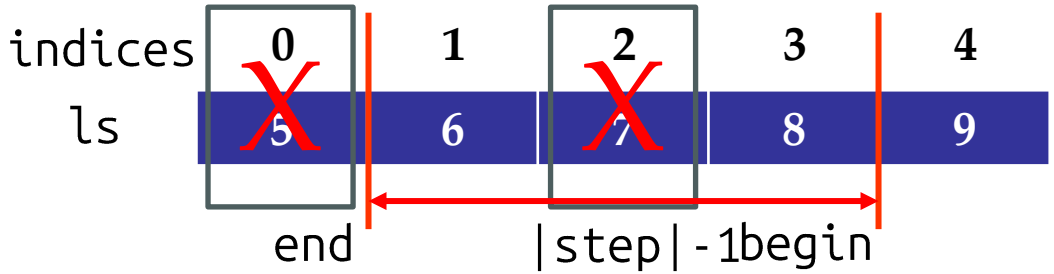


© MITB, SMU

List operations – index operations

- Slice a subset of a list – with negative step (from right to left)
  - `list[begin:end:step]`: From index `begin` to index `end`, at a `step` size, inclusive of `begin`, but exclusive of `end`

```
>>> ls = [5, 6, 7, 8, 9]
>>> ls[3:0:-2]
[8, 6]
```



- Reversing a list `ls` can simply be done using `ls[::-1]`

© MITB, SMU

Wooclap!

What is the output of the below print statement?

```
ls = [1, 2, 3, 4, 5]
print(ls[::-1])
```

a. [1, 2, 3, 4, 5]

b. [5, 4, 3, 2, 1]

c. [1, 2, 3, 4]

List operations – search operations

- `index` Find the **index** of the **first** occurrence of the value.
  - `list.index(query_value, start_index, end_index)`
  - `start_index` and `end_index` specifies where to begin and end the search, respectively
  - Inclusive of `start_index`, but exclusive of `end_index`
  - Returns an error when value is not in the list

```
>>> ls = [1, 2, 3, 4, 5, 1, 2, 3, 4, 5]
>>> ls.index(4, 5, 9)
8
>>> ls.index(8)
Traceback (most recent call last):
  File '<stdin>', line 1, in <module>
ValueError: 8 is not in list
```


© MITB, SMU

List operations – insert operations

- append Inserts a **single** item at the **tail** location
  - list.append(item)

if you append a list to a list, you get a list of a list

ls = ['p', 'q']
ls.append('t')
ls = ['p', 'q', 't']
ls.append(['v'])
ls = ['p', 'q', 't', ['v']]

```
>>> ls = [1, 2, 3, 4, 5]
>>> ls.append(6)
# ls is now [1, 2, 3, 4, 5, 6]
```

- insert Inserts a **single** item at the **index** location
  - list.insert(index, item)

```
>>> ls.insert(1, 9)
# ls is now [1, 9, 2, 3, 4, 5, 6]
```

- extend Inserts **multiple** items at the **tail** location
  - list.extend(items)

combines 2 lists together into 1 list

```
>>> ls.extend([1, 2])
# ls is now [1, 9, 2, 3, 4, 5, 6, 1, 2]
```


© MITB, SMU

List operations – delete operations

- `remove` Removes a **single** item, starting from the front of the list
  - `list.remove(item)`

↳ the specific value/item that you want to remove

```
>>> ls = [1, 2, 3, 4, 5]
>>> ls.remove(3)
# ls is now [1, 2, 4, 5]
```

ls = [1, 3, 5]
ls.remove(5)
ls = [1, 3]

- `pop` Removes at the **index** location (default is -1) and returns value
  - `list.pop(index)`

- `clear` **Empties** the list
  - `list.clear()`

```
>>> ls.pop()
5
# ls is now [1, 2, 4]
>>> ls.pop(0)
1
# ls is now [2, 4]
```


© MITB, SMU

List operations – sorting

- `sort` Arranges in ascending order
  - `list.sort()`
  - Using `list.sort(reverse=True)` instead sorts in descending order

```
>>> ls = [3, 1, 2, 5, 4]
>>> ls.sort()
# ls is now [1, 2, 3, 4, 5]
>>> ls.sort(reverse=True)
# ls is now [5, 4, 3, 2, 1]
```


© MITB, SMU

What about Replace?

- You can directly replace values in the list through their index.
  - `list[pos_to_replace] = value_to_replace`

```
>>> ls = [1, 2, 3, 4, 5]
>>> ls[3] = 6
# ls is now [1, 2, 3, 6, 5]
```


© MITB, SMU

Combining Strings and Lists

- `split` Convert from string to list using a `delimiter`
  - `string.split(delimiter)`

```
>>> s = 'mitb.smu.edu.sg'
>>> print(s.split('.'))
['mitb', 'smu', 'edu', 'sg']
```

- `join` Use a string to connect multiple strings in a list
  - `string.join(list_of_strings)`
  - Same as printing using sep

```
>>> ls = ['mitb', 'smu', 'edu', 'sg']
>>> print('.'.join(ls))
'mitb.smu.edu.sg'
>>> print(ls[0], ls[1], ls[2], ls[3], sep='.')
'mitb.smu.edu.sg'
```


© MITB, SMU

Wooclap!

What is the output of the below print statement?

```
s = "hahahahaha"
print(s.split("a"))
```

a. ['h', 'h', 'h', 'h', 'h', '']

b. ['h', 'h', 'h', 'h', 'h']

The reverse is:

ans = ['h', 'h', 'h', 'h', '']

'a'.join(ans) . ⇒ returns 'hahahaha'

TAKE NOTE:

this not

ans.join('a')

Dictionaries

- A zookeeper wishes to manage the quantity of animals in his zoo. E.g., he wants to store and lookup how many tigers, goats, and hawks he has.

- How can he do this using lists?

A list to remember the animals to be counted

0	1	2	3	4	5	6	7	8	9
deer	goat	wolf	boar	hare	puma	hawk	pony	bear	lion



Another list to remember the values of the counters

0	1	2	3	4	5	6	7	8	9
8	11	15	19	34	4	2	6	2	2

© MITB, SMU

Dictionaries

A list to remember the animals to be counted

0	1	2	3	4	5	6	7	8	9
deer	goat	wolf	boar	hare	puma	hawk	pony	bear	lion



Another list to remember the values of the counters

0	1	2	3	4	5	6	7	8	9
8	11	15	19	34	4	2	6	2	2

- Given this, he would require a complicated loop / use index to search for the quantity of an animal

- Is it more convenient to combine the two lists?

deer	goat	wolf	boar	hare	puma	hawk	pony	bear	lion
8	11	15	19	34	4	2	6	2	2

© MITB, SMU

Dictionaries

- A list is an easy-to-use container, but it requires all cells to be indexed from 0

Dictionary is a HashMap.

- Dictionaries allows indexing using other data types (e.g., strings)

- In a dictionary, we represent data using key-value pairs

deer	goat	wolf	...
8	11	15	...

← Keys

← Values

Key-value pairs.

- Keys must be unique, but values may not

© MITB, SMU

Dictionary operations – creating a dict

- Dictionaries can be created manually using
  `{key1: value1, key2: value2, …}`

```
>>> d1 = {1: 2, 3: 4}
```

- You can also use strings as keys

```
>>> d2 = {'deer': 8, 'goat': 11, 'wolf': 15}
```

- Values of a dictionary need not be numbers and need not be of the same type.

```
>>> d3 = {'a': 1, 'b': 'hello', 'c': ['a', 1, [0,1]], 'd': {1: 2}}
```


© MITB, SMU

Dictionary operations – index operations

- `len(d)`: gives the length of dictionary `d`

```
>>> d = {'deer': 8, 'goat': 11, 'wolf': 15}
>>> len(d)
3
```

deer	goat	wolf
8	11	15

`len(d) = 3`

- Access values of a dictionary by their keys, indexes no longer work here
  - `d[key]` Returns the value paired with the corresponding `key`
  - Throws an error if `key` is not in `d`

```
>>> d = {'deer': 8, 'goat': 11, 'wolf': 15}
>>> d['wolf']
15
>>> d['tiger']
KeyError: 'tiger'
```

→ will block the rest of the code from running. use d.get('tiger') instead.

© MITB, SMU

Dictionary operations – index operations

- `d.get(key, default_value)`: Returns the value paired with the corresponding `key`
  - Returns the `default_value` if `key` is not in `d`

deer	goat	wolf
8	11	15

```
>>> d = {'deer': 8, 'goat': 11, 'wolf': 15}
>>> d.get('wolf')
15
>>> d.get('tiger', 0)
0
```


© MITB, SMU

Dictionary operations – insert/update

- Inserting or updating values of a dictionary are done the same way
  - `d[key] = value` Inserts / updates the `value` corresponding to the `key` value.

```
>>> d = {'deer': 8, 'goat': 11, 'wolf': 15}
>>> d['tiger'] = 5
# d is now {'deer': 8, 'goat': 11, 'wolf': 15, 'tiger': 5}
>>> d['deer'] = 3
# d is now {'deer': 3, 'goat': 11, 'wolf': 15, 'tiger': 5}
```


© MITB, SMU

Dictionary operations – delete operations

- To delete a key-value pair from a dictionary, either use
  - `d.pop(key)` Deletes the key-value pair corresponding to the key value. Returns the deleted value.
  - `del d[key]` Deletes the key-value pair corresponding to the key value. Does not return the deleted value.

```
>>> d = {'deer': 8, 'goat': 11, 'wolf': 15}
>>> print(d.pop('wolf'))
15
# d is now {'deer': 8, 'goat': 11}
>>> del d['deer']
# d is now {'goat': 11}
```


© MITB, SMU

Membership Operator

- `in` Checks whether an item is inside a container
  - `item in container`
  - If `container` is a list, checks whether `item` is inside the list
  - If `container` is a dictionary, checks whether `item` is a **key** in the dictionary
- `not in` Checks whether an item is **not** inside a container
  - `item not in container`

```
>>> ls = [1, 2, 3, 4, 5]
>>> d = {'deer':8, 'goat':11, 'wolf':15}
>>> print(3 in ls)
True
>>> print('goat' in d)
True
>>> print('tiger' in d)
False
>>> print(6 not in ls)
True
```


© MITB, SMU

Dictionary operations – keys, values, items


```
>>> d = {'deer': 8, 'goat': 11, 'wolf': 15}
```

- `d.keys()` Returns all the **keys** in the dictionary

```
>>> print(list(d.keys()))
>>> ['deer', 'goat', 'wolf']
```

- `d.values()` Returns all the **values** in the dictionary

```
>>> print(list(d.values()))
>>> [8, 11, 15]
```

- `d.items()` Returns all the **key-value** pairs in the dictionary

```
>>> print(list(d.items()))
>>> [('deer', 8), ('goat', 11), ('wolf', 15)]
```

- All can be casted into list type, e.g., `list(d.items())`

© MITB, SMU

Wooclap!

What is the output of the below print statement?

```
d = {'a': 4, 'b': 2, 'c': 3, 'd': 1}
for k, v in d.items():
    if v % 2 == 0:
        v = v - 2
    else:
        d[k] = 4
print(d)
```

a. {'a': 4, 'b': 2, 'c': 3, 'd': 1}

b. {'a': 2, 'b': 0, 'c': 3, 'd': 1}

c. {'a': 4, 'b': 2, 'c': 4, 'd': 4}

d. {'a': 2, 'b': 0, 'c': 4, 'd': 4}

Tuples

- **Fixed size** sequence of objects that are immutable
- Immutable objects cannot be changed after it is created
- Similar to lists!

  `tuple` : **fixed** size sequences of objects, **immutable**

  – A bundle of several constants

```
>>> orange_peel = (1.0, 0.625, 0.0)
```

  `list` : **dynamic** size sequences of objects, **mutable**

  – Easy to access, slice, add, delete or replace elements
- Lists are much more popular than tuples

  – List is ~~one of~~ the most common data structures in Python

© MITB, SMU

Tuple operations – creating a tuple

- Tuples can be created manually using `()`.

```
>>> tp = (1, 2, 3, 4, 5)
```

- `tuple()` can also be used to create tuples from other sequences.

```
>>> tp = tuple(range(1, 6))
# creates (1, 2, 3, 4, 5)
```

- Elements of a tuple need not be numbers and need not be of the same type.

```
>>> tp = (1, '2', True, 'hello', (1, 2))
```


© MITB, SMU

Sets

- Can be considered as "keys only dictionaries"
- Elements must be unique
- `len` the size of the set
- `add` add one element
- `update` union another list
- `pop` pop an element out
- `remove` remove a specific element

```
>>> s = {'deer', 'hawk', 'wolf', 'boar'}
>>> s.add('bear')
>>> s.add(('goat', 'pony', 'hare'))
>>> s.update(['lion', 'puma', 'tiger', 'leopard'])
>>> print(s)
{'puma', 'lion', 'bear', ('goat', 'pony', 'hare'),
'leopard', 'deer', 'wolf', 'hawk', 'tiger', 'boar'}
>>> s.pop()
'puma'
>>> s.remove('leopard')
>>> print(s)
{'lion', 'bear', ('goat', 'pony', 'hare'), 'deer',
'wolf', 'hawk', 'tiger', 'boar'}
```

non-examinable.

Optional Reading

© MITB, SMU

for Loops with iterables

- An iterable is any Python object which can be looped over or iterated over with the help of a for loop

- Python Containers are iterables:
  - List
  - Dictionaries
  - Tuples
  - Sets

- Other Examples:
  - Range (seen earlier in for loops)
  - String

© MITB, SMU

for Loops with iterables

- You can use python containers as an iterator in for loops.
  - `for v in list:`
    `    do something for each value v`
  - `for k, v in dictionary.items():`
    `    do something for each pair k, v`
  - `for e in set:`
    `    do something for each element e`

```
>>> for v in l:
...     print(v)
>>> for i in range(len(l)):
...     print(l[i])
>>> for k in d:
...     print(k, d[k])
>>> for k, v in d.items():
...     print(k, v)
```

- You can also use strings as an iterator in for loops.
  - `for a in string:`
    `    do something for each value a`

```
>>> s = 'abcdef'
>>> for a in s:
...     print(a)
```


© MITB, SMU

Logical operators

- Boolean operators
  - `any`: Return True if any element of the iterable is true.
  - `all`: Return True if all elements of the iterable are true, or if the iterable is empty

```
>>> any([False, True, False, False])
True
>>> all([False, True, True, False])
False
```

ANY table	
[False, True, False, False]	True
[False, False, False, False]	False
[True, True, True]	True
[]	False

ALL table	
[True, True, True, True]	True
[False, True, True, False]	False
[False, False, False]	False
[]	True

© MITB, SMU

Checkpoint

- When do you use lists?
- When do you use dictionaries?
- When do you use tuples?
- When do you use sets?

© MITB, SMU

Exercise 2-2

The calculation of income tax rates depends on an individual's tax residency status. Singapore's personal income tax rates for resident taxpayers are progressive. This means that higher income earners pay a proportionately higher tax, with the current highest personal income tax rate at 24%.

Write a Python code to compute the resident tax given a certain accessible income.

© MITB, SMU

Exercise 2-2

From YA 2024 onwards

Chargeable Income	Income Tax Rate (%)	Gross Tax Payable ($)
First $20,000 Next $10,000	0 2	0 200
First $30,000 Next $10,000	- 3.50	200 350
First $40,000 Next $40,000	- 7	550 2,800

As of the year 2024, the resident tax rates are as follows:

Source: https://www.iras.gov.sg/taxes/individual-income-tax/basics-of-individual-income-tax/tax-residency-and-tax-rates/individual-income-tax-rates

- How do we get started?
  - Interpreting the data
  - Designing the data structure to store data

© MITB, SMU

Exercise 2-2 (Solution)

- One way to translate the table
- A table format that is more commonly seen

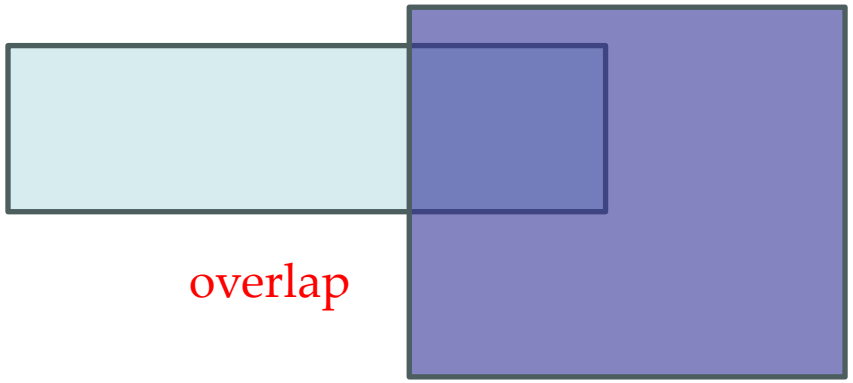
Assessible Income	Income Tax Rate (%)
First $20000	0
Next $10000	2
Next $10000	3.5
Next $40000	7
Next $40000	11.5
Next $40000	15
Next $40000	18
Next $40000	19
Next $40000	19.5
Next $40000	20
Next $180000	22
Next $500000	23
Remaining	24

© MITB, SMU

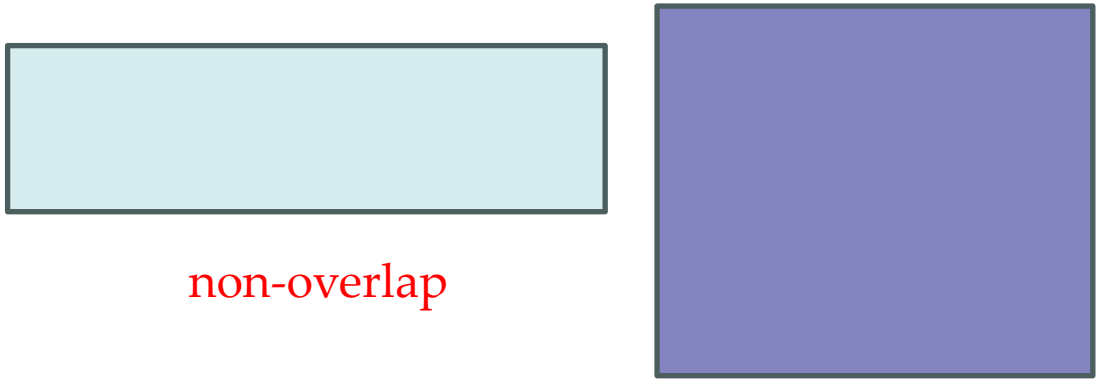
Exercise 2-3

A rectangle in 2D can be represented by their x and y coordinates of two diagonally opposite points (usually the top left and bottom right corners).

In this problem, you are given two rectangles (their top left and bottom right coordinates). Your objective is to calculate whether the two rectangles intersect (i.e. overlap each other).



overlap

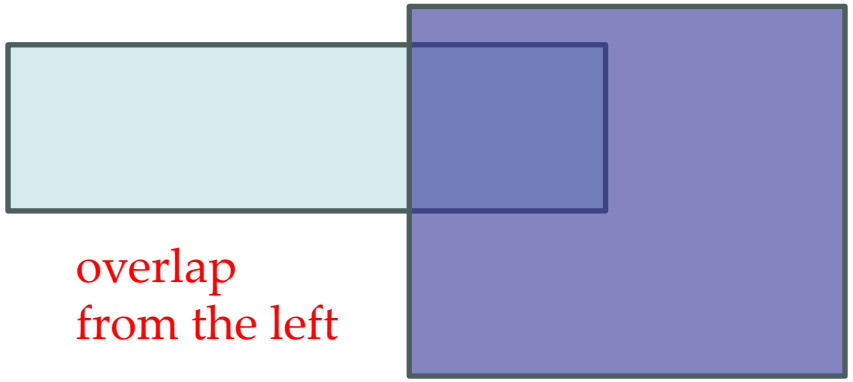


non-overlap

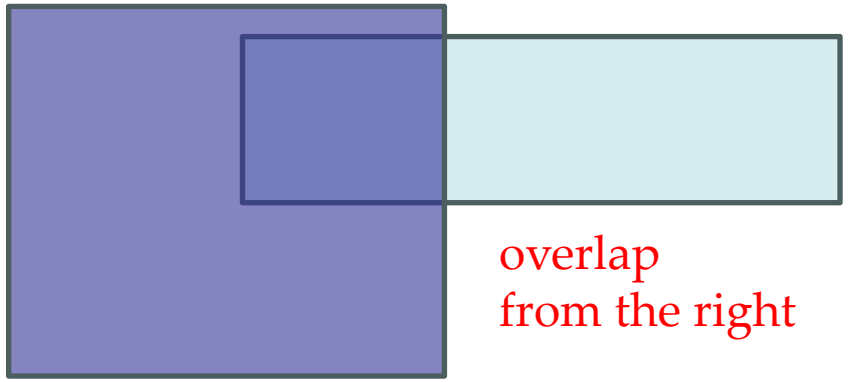
© MITB, SMU

Exercise 2-3 (Naïve Solution)

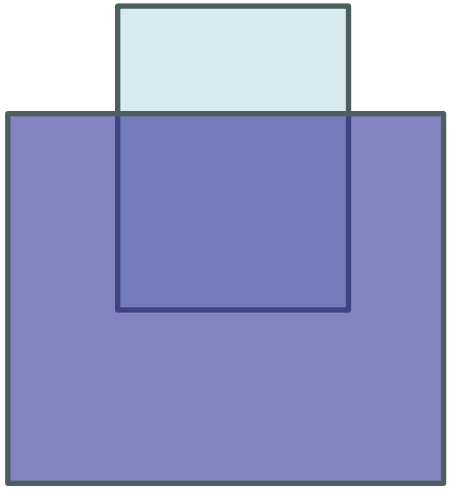
- Let us try to list out **all** the possible cases in which they overlap



overlap
from the left



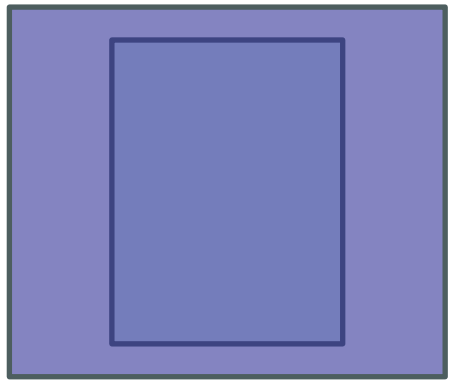
overlap
from the right



overlap
from the top



overlap
from the bottom



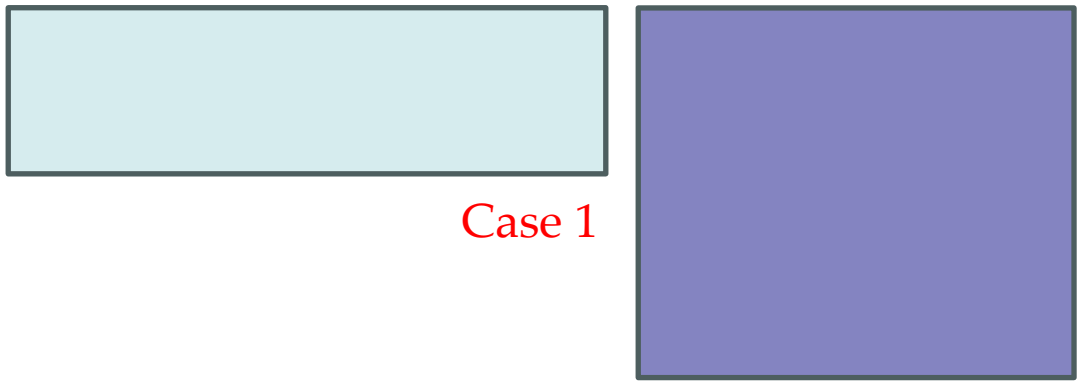
overlap fully

AND MANY MORE!

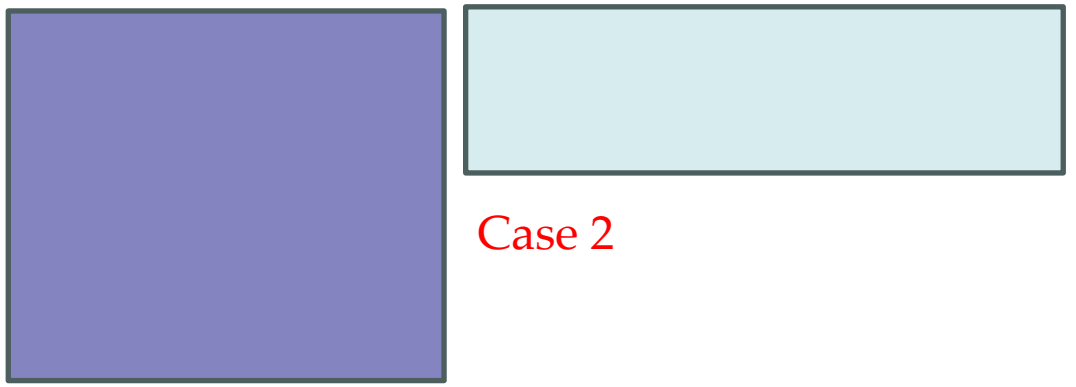
© MITB, SMU

Exercise 2-3 (Decomposition: Cases) easier way

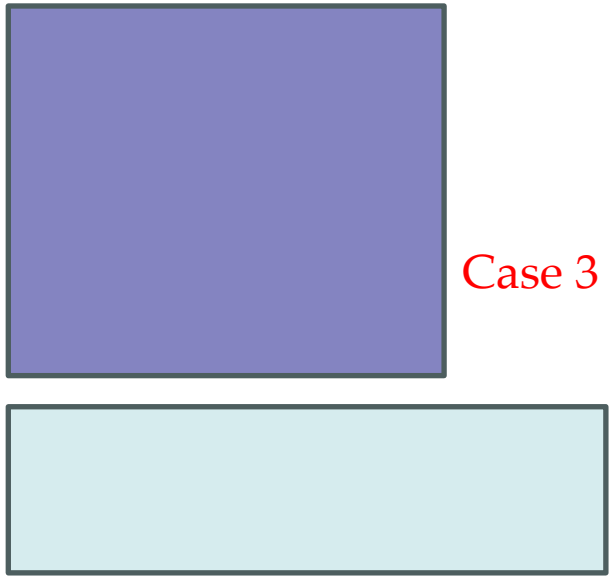
- There are a total of 4 cases in which they **do not** overlap



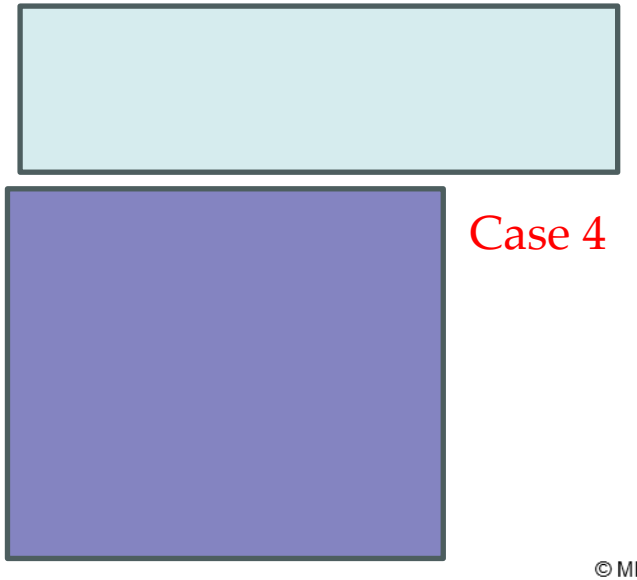
Case 1



Case 2



Case 3

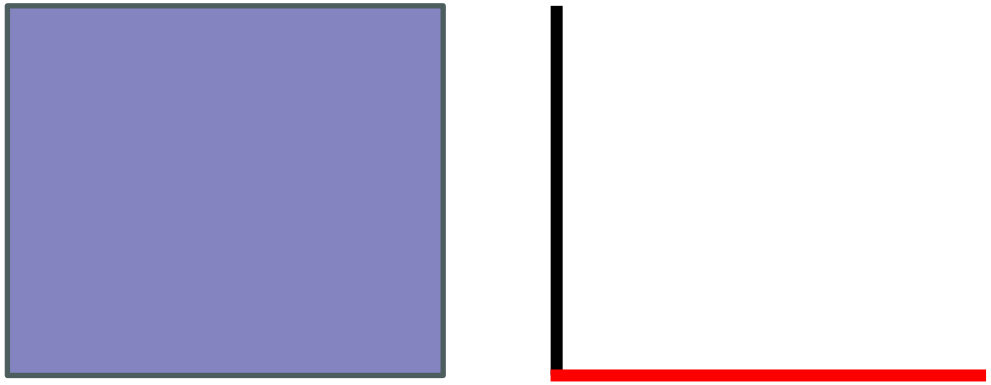


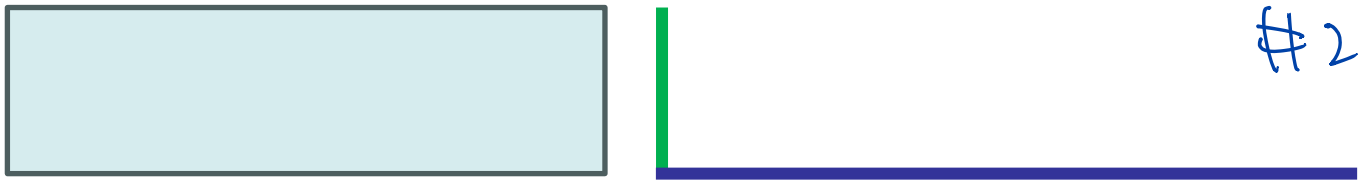
Case 4

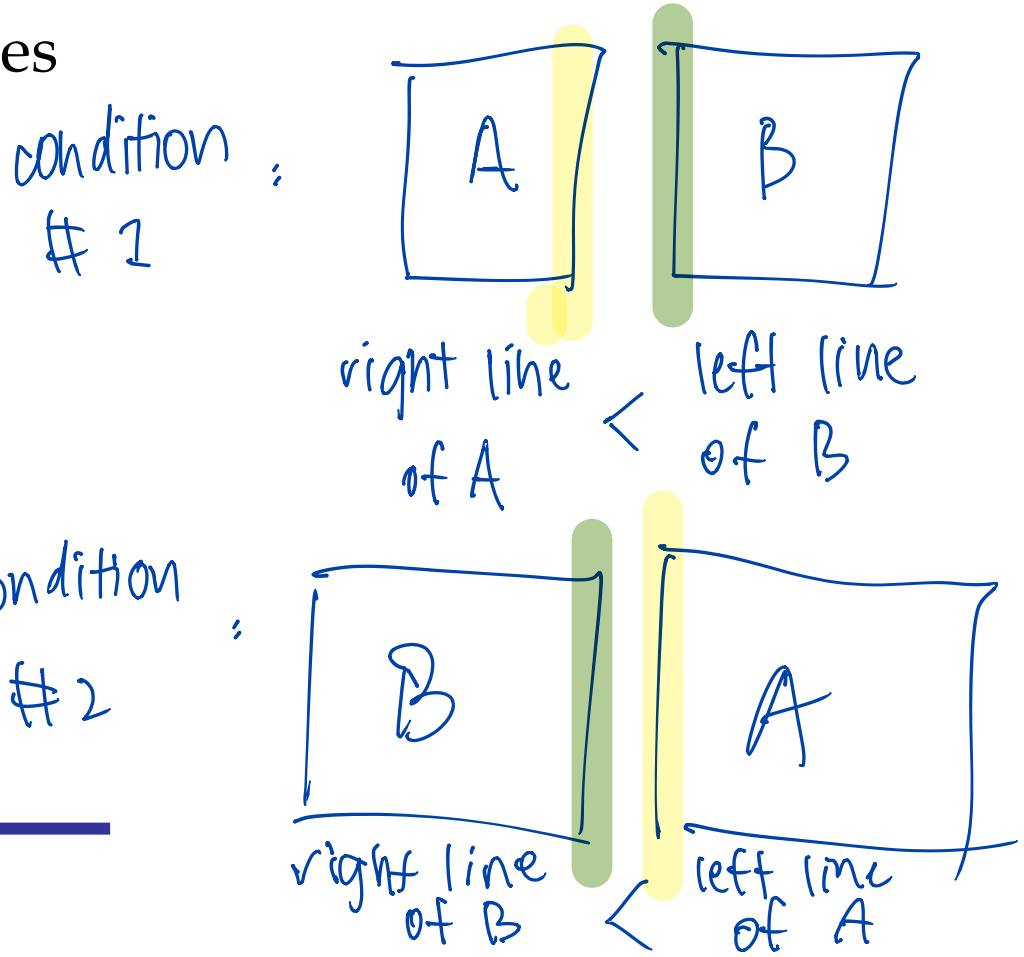
© MITB, SMU

Exercise 2-3 Solution (Decomposition)

- Let us break down the problem into an easier sub-problem
- A rectangle can be fully represented by 2 lines



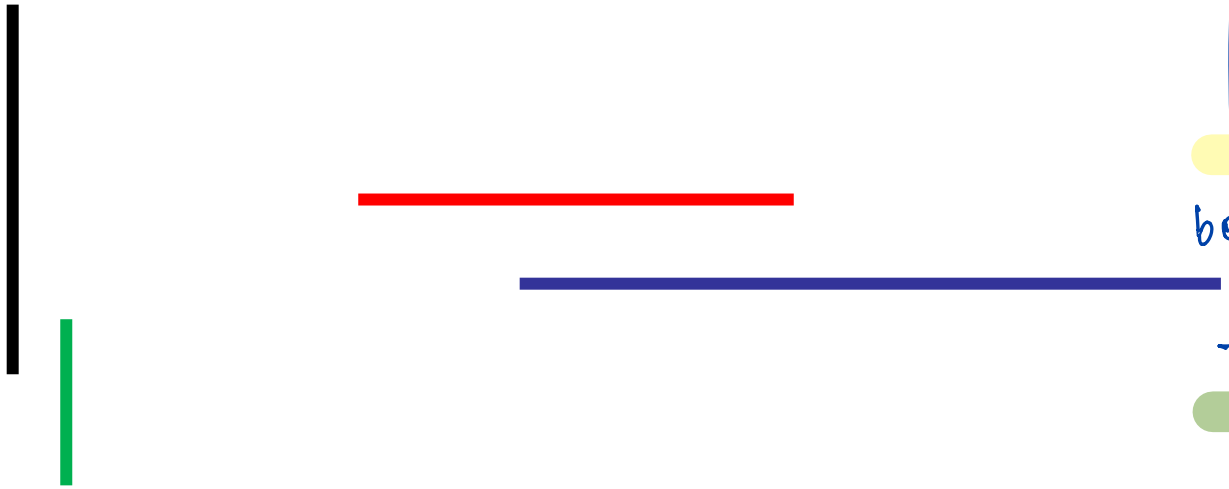


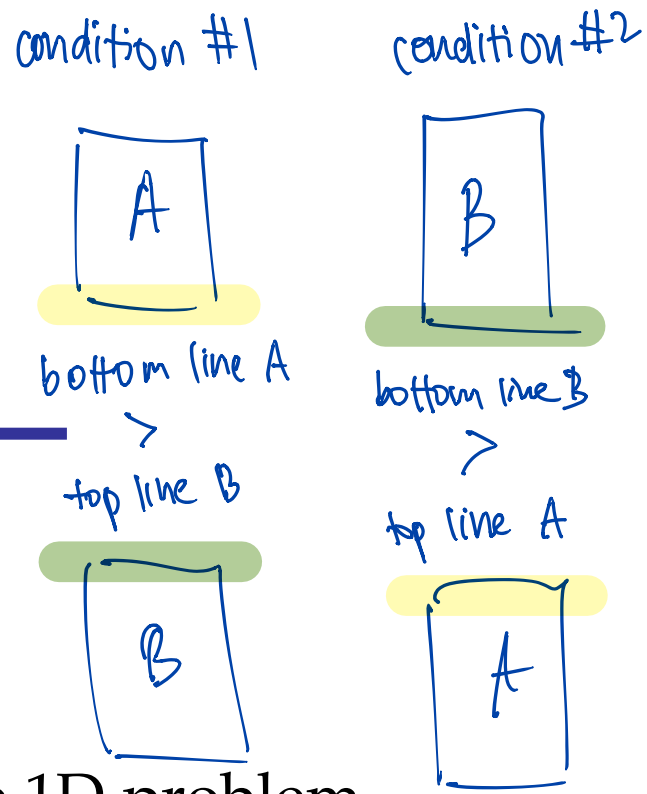


© MITB, SMU

Exercise 2-3 Solution (Decomposition)

- To check whether the 2 rectangles intersect, we simply need to check whether **both** the parallel lines intersect in 1D!





- We have now broken down the 2D problem to solving the 1D problem

© MITB, SMU

Exercise 2-3 Solution (Decomposition)

- Breaking down 2D problems to 1D problems usually work, because when working with smaller dimensions, the number of cases you need to check decreases drastically
- E.g., now, to check if two lines **do not** intersect in 1D, we can just naively check 2 conditions, compared to 4 in the 2D case



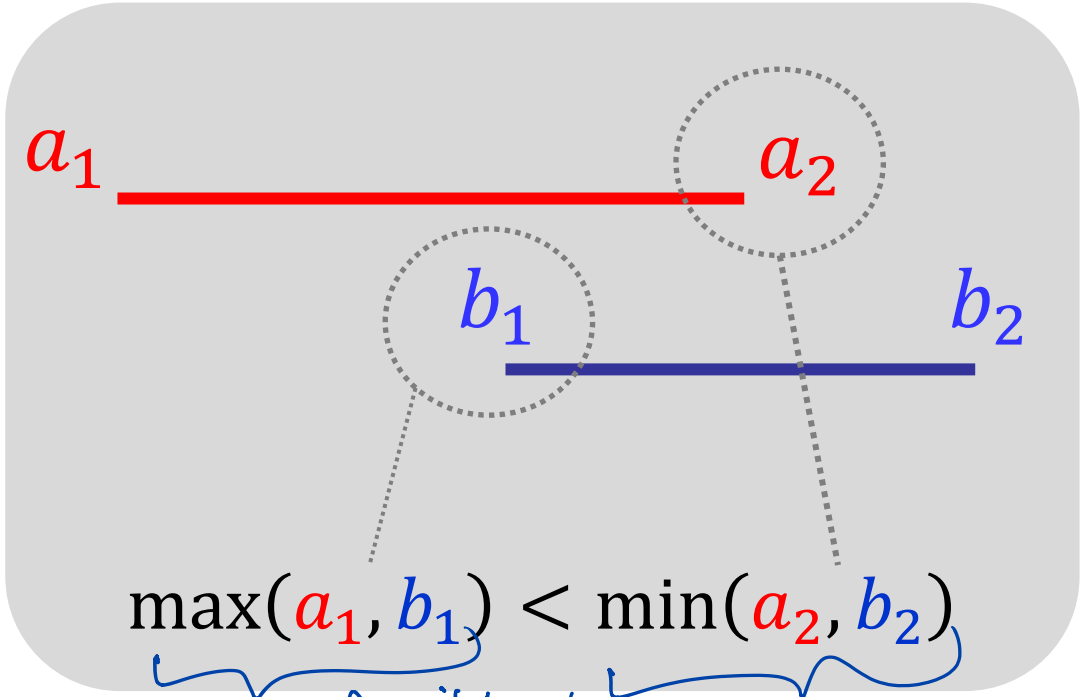
Case 1 Case 2

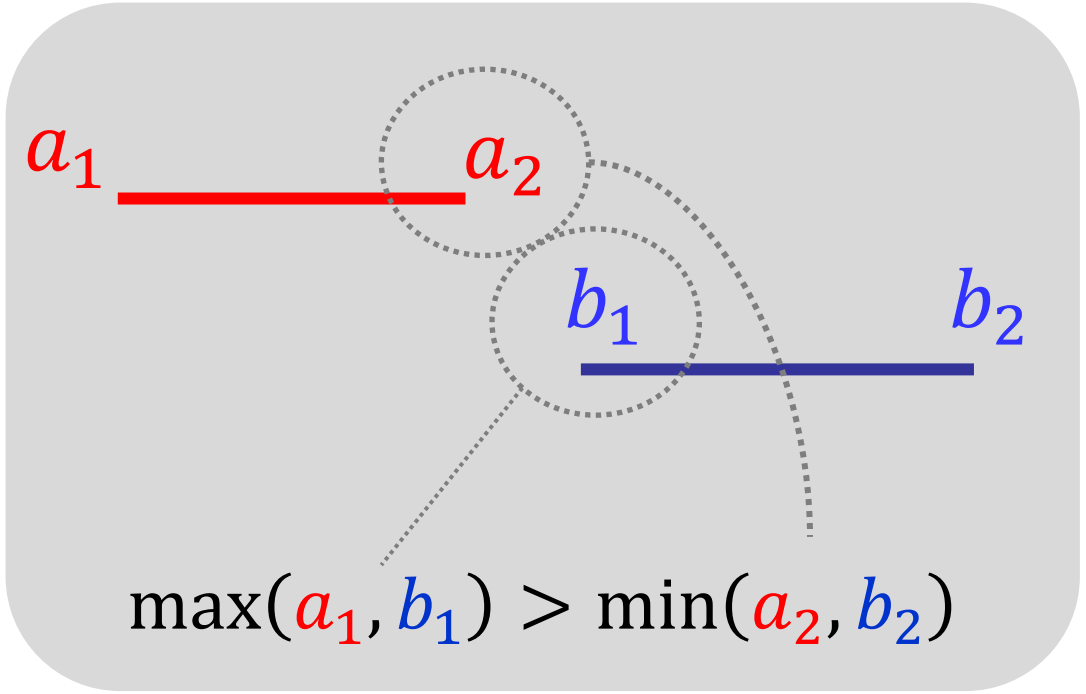
- Try thinking how many such cases there will be for a 3D cuboid!

© MITB, SMU

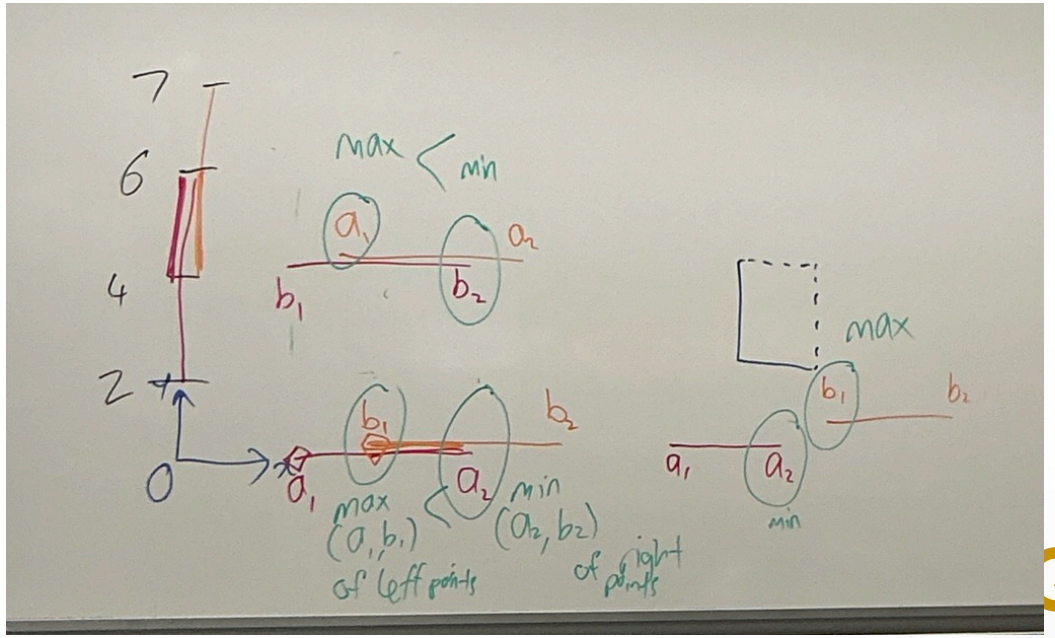
Exercise 2-3 Solution (Decomposition)

- In fact, there is even an easier way to check whether two lines overlap!





© MITB, SMU



Questions?

Thank You

© MITB, SMU